Lab 6 Report — Resilient WAN and Secure Microservice Exposure

Course

CP352005 Computer Networks
Khon Kaen University

Authors

673380416-3 นายพัทธดนย์ คำนัน

673380428-6 นายสิทธิโชค มุขนาค

673380583-4 นายณัฐภัทร ฉ่ำตะคุ

673380606-8 นายสรวิศ สุคงเจริญ

673380608-4 นางสาวอมลวรรณ พิมพิชัย

GitHub Repo: https://github.com/ms584/CP352005-Computer-Networks-Laboratory.git

1. Introduction

Laboratory 6 extends the SME network architecture developed in the previous laboratories by introducing enterprise-level networking features including secure service exposure, network address translation (NAT), firewall policy enforcement, and WAN monitoring.

The objective of this laboratory is to simulate a realistic enterprise network environment where internal microservices remain protected while only specific services are exposed to the public internet. The lab environment is implemented using Docker containers to emulate routers, servers, and client systems across multiple network segments.

This lab focuses on improving the following aspects of network infrastructure:

- Secure service exposure
- NAT-based Internet connectivity
- Firewall policy enforcement
- WAN health monitoring
- Microservice isolation

2. Network Architecture

The simulated network consists of two LAN segments connected through routers and an ISP network.

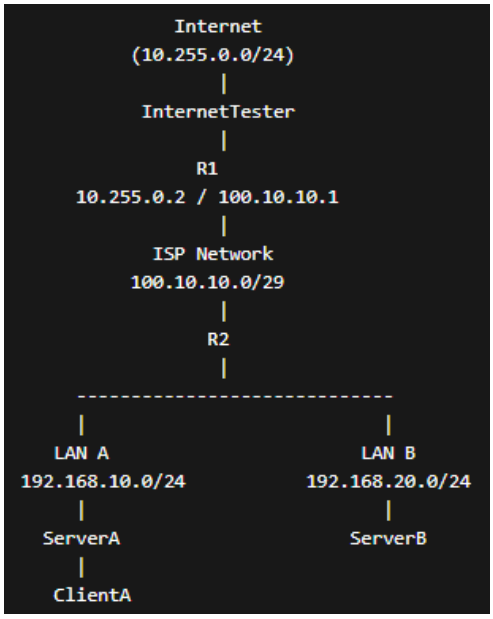
Network Segments

Network	Subnet
Internet	10.255.0.0/24
ISP Network	100.10.10.0/29
LAN A	192.168.10.0/24
LAN B	192.168.20.0/24

Infrastructure Components

Device	Role
R1	Edge Router (NAT + Firewall)
R2	Internal Router
ServerA	Public microservice host
ServerB	Internal microservice host
ClientA	Internal client
InternetTester	Simulated Internet client

Network Topology



3. Infrastructure Deployment

The entire infrastructure was deployed using Docker Compose.

Start the Environment

```
docker compose down -v
docker compose up --build -d
```

Verify running containers:

```
docker ps
```

Expected containers: R1, R2, ServerA, ServerB, ClientA, InternetTester

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS         PORTS     NAMES
bc5bfc27018e   python:3.10-alpine "sh -c 'apk add --no…"   9 seconds ago  Up 8 seconds             ServerA
f238e01c21c2   networklab6-r2     "/bin/sh /start.sh"      9 seconds ago  Up 8 seconds             R2
f4f5c41a7e12   python:3.10-alpine "sh -c 'apk add --no…"   9 seconds ago  Up 8 seconds             ServerB
0bca158bcdd1   networklab6-r1     "/bin/sh /start.sh"      9 seconds ago  Up 8 seconds             R1
679a25943b83   python:3.10-alpine "sh -c 'apk add --no…"   9 seconds ago  Up 8 seconds             InternetTester
6a2c47872ca6   python:3.10-alpine "sh -c 'apk add --no…"   9 seconds ago  Up 8 seconds             ClientA
81f4437c4019   python:3.10-alpine "sh -c 'apk add --no…"   9 seconds ago  Up 8 seconds             ClientB
```

4. Microservice Deployment

Microservices were deployed on both server nodes.

Start services

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -d ServerA sh -c "python /automation/start_services.py"

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -d ServerB sh -c "python /automation/start_services.py"
```

Verify running processes:

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec ServerA ps
PID   USER     TIME  COMMAND
    1 root      0:00 python /automation/start_services.py
   18 root      0:00 python /automation/start_services.py
   28 root      0:00 ps

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec ServerA ps
PID   USER     TIME  COMMAND
    1 root      0:00 python /automation/start_services.py
   18 root      0:00 python /automation/start_services.py
   34 root      0:00 ps
```

Verify open service ports

ServerA

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec ServerA ss -tuln
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
udp   UNCONN 0      0         127.0.0.11:40518      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:9000       0.0.0.0:*
tcp   LISTEN 0      4096      127.0.0.11:39969      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8000       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8001       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8002       0.0.0.0:*
```

ServerB

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec ServerB ss -tuln
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
udp   UNCONN 0      0         127.0.0.11:39685      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8002       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8000       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8001       0.0.0.0:*
tcp   LISTEN 0      4096      127.0.0.11:46479      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:9000       0.0.0.0:*
```

Expected services:

Port	**Service**
8000	Upload Service
8001	Processing Service
8002	AI Service
9000	Gateway Service

5. Internal Network Connectivity Test

Connectivity between LAN segments was verified.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it ClientA ping -c 3 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=62 time=0.097 ms
64 bytes from 192.168.20.10: seq=1 ttl=62 time=0.075 ms
64 bytes from 192.168.20.10: seq=2 ttl=62 time=0.087 ms

--- 192.168.20.10 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.075/0.086/0.097 ms
```

Expected result:

```
0% packet loss
```

This confirms that routing between LAN A and LAN B is functioning correctly.

6. Internal Service Access Test

ClientA accessed services hosted on ServerB.

Health check service

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it ClientA sh -c "curl -v http://192.168.20.10:8000/health; echo"
*   Trying 192.168.20.10:8000...
* Established connection to 192.168.20.10 (192.168.20.10 port 8000) from 192.168.10.50 port 36496
* using HTTP/1.x
> GET /health HTTP/1.1
> Host: 192.168.20.10:8000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Wed, 11 Mar 2026 16:39:21 GMT
< Content-type: application/json
<
* shutting down connection #0
```

Expected response:

```
* HTTP 1.0, assume close
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6
```

Workflow service

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it ClientA sh -c "curl -v -X POST http://192.168.20.10:9000/process-file; echo"
*   Trying 192.168.20.10:9000...
* Established connection to 192.168.20.10 (192.168.20.10 port 9000) from 192.168.10.50 port 52258
* using HTTP/1.x
> POST /process-file HTTP/1.1
> Host: 192.168.20.10:9000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Wed, 11 Mar 2026 16:40:34 GMT
< Content-type: application/json
<
* shutting down connection #0
```

Expected response:

```
* Request completely sent of
* HTTP 1.0, assume close aft
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Pyt
< Date: Wed, 11 Mar 2026 16:
```

7. Internet Connectivity Test

ServerA tested outbound Internet connectivity through NAT.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it ServerA ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=62 time=16.124 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=14.898 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=14.642 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 14.642/15.221/16.124 ms
```

Expected output:

```
0% packet loss
124 ms
```

This confirms that outbound NAT translation is functioning correctly.

8. Routing Verification

Router routing tables were inspected.

R1 Routing Table

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 ip route
default via 10.255.0.1 dev eth0
8.8.8.8 via 10.255.0.1 dev eth0
10.255.0.0/24 dev eth0 proto kernel scope link src 10.255.0.2
100.10.10.0/29 dev eth1 proto kernel scope link src 100.10.10.1
192.168.10.0/24 dev eth2 proto kernel scope link src 192.168.10.1
192.168.20.0/24 nhid 14 via 100.10.10.2 dev eth1 proto ospf metric 20
```

Expected entries:

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 ip route
default via 10.255.0.1 dev eth0
8.8.8.8 via 10.255.0.1 dev eth0
10.255.0.0/24 dev eth0 proto kernel scope link src 10.255.0.2
100.10.10.0/29 dev eth1 proto kernel scope link src 100.10.10.1
192.168.10.0/24 dev eth2 proto kernel scope link src 192.168.10.1
192.168.20.0/24 nhid 14 via 100.10.10.2 dev eth1 proto ospf metric 20
```

R2 Routing Table

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R2 ip route
default via 100.10.10.6 dev eth0
100.10.10.0/29 dev eth0 proto kernel scope link src 100.10.10.2
192.168.10.0/24 nhid 12 via 100.10.10.1 dev eth0 proto ospf metric 20
192.168.20.0/24 dev eth1 proto kernel scope link src 192.168.20.1
```

Expected entries:

```
default via 100.10.10.6 dev eth0
100.10.10.0/29 dev eth0 proto kernel scope link src 100.10.10.2
192.168.10.0/24 nhid 12 via 100.10.10.1 dev eth0 proto ospf metric 20
192.168.20.0/24 dev eth1 proto kernel scope link src 192.168.20.1
```

9. NAT Configuration Verification

NAT configuration was verified using iptables.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       6    --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt:8000 to:192.168.10.10:8000

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
  165 13860 MASQUERADE  0    --  *      eth0    0.0.0.0/0            0.0.0.0/0
    0     0 MASQUERADE  6    --  *      eth2    0.0.0.0/0            192.168.10.10        tcp dpt:8000

Chain DOCKER_OUTPUT (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER_POSTROUTING (0 references)
 pkts bytes target     prot opt in     out     source               destination
```

Observed rule:

```
ation
0/0            tcp dpt:8000 to:192.168.10.10:8000

ation
```

DNAT tcp dpt:8000 -> 192.168.10.10:8000
MASQUERADE outbound traffic

This rule allows only port 8000 to be exposed publicly.

10. Firewall Policy Verification

Firewall rules were inspected.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
   28  2229 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
    0     0 ACCEPT     6    --  eth0   eth2    0.0.0.0/0            192.168.10.10        tcp dpt:8000
    1    84 ACCEPT     0    --  eth2   eth0    192.168.10.0/24      0.0.0.0/0
    3   204 ACCEPT     0    --  eth2   eth1    192.168.10.0/24      192.168.20.0/24
    0     0 ACCEPT     0    --  eth1   eth2    192.168.20.0/24      192.168.10.0/24
    0     0 DROP       0    --  eth0   eth2    0.0.0.0/0            0.0.0.0/0
    0     0 DROP       0    --  eth1   eth2    0.0.0.0/0            0.0.0.0/0
```

Firewall policy:

Rule	Description
Allow established connections	return traffic
Allow Internet → ServerA:8000	public service
Allow LAN outbound traffic	internal access
Drop other inbound connections	security protection

11. Public Service Exposure Test

Internet access was simulated using the InternetTester container.

Public service (allowed)

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it InternetTester sh -c "curl -v http://10.255.0.2:8000/health; echo"
*   Trying 10.255.0.2:8000...
* Established connection to 10.255.0.2 (10.255.0.2 port 8000) from 10.255.0.10 port 49450
* using HTTP/1.x
> GET /health HTTP/1.1
> Host: 10.255.0.2:8000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Wed, 11 Mar 2026 16:49:22 GMT
< Content-type: application/json
<
* shutting down connection #0
```

Result:

```
>
* Request completely sent off
* HTTP 1.0, assume close after b
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/
< Date: Wed, 11 Mar 2026 16:49:2
```

Backend service (blocked)

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec -it InternetTester sh -c "curl -v http://10.255.0.2:9000/process-file; echo"
*   Trying 10.255.0.2:9000...
* connect to 10.255.0.2 port 9000 from 10.255.0.10 port 35636 failed: Connection refused
* Failed to connect to 10.255.0.2 port 9000 after 0 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to 10.255.0.2 port 9000 after 0 ms: Could not connect to server
```

Result:

Connection refused

This confirms that backend services remain inaccessible from the public network.

12. Connection Tracking Verification

Connection tracking entries were inspected.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 cat /proc/net/nf_conntrack | grep 8000
ipv4     2 tcp      6 14 TIME_WAIT src=10.255.0.10 dst=10.255.0.2 sport=49450 dport=8000 src=192.168.10.10 dst=192.168.10.1 sport=8000 dport=49450 [ASSURED] mark
=0 zone=0 use=2
```

Example entry:

src=10.255.0.10 dst=10.255.0.2 dport=8000
dst=192.168.10.10

This confirms that NAT translation occurred correctly.

13. WAN Monitoring

The WAN monitoring mechanism verifies Internet reachability using a persistent route to a public IP address.

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 6 (main)
$ docker exec R1 ip route | grep 8.8.8.8
8.8.8.8 via 10.255.0.1 dev eth0
```

Expected result:

8.8.8.8 via 10.255.0.1

This route ensures that WAN monitoring remains functional even if the default route changes.

14. Security Analysis

The network implements a secure microservice architecture with the following protections:

Feature	Description
NAT	hides internal network addresses
DNAT	selectively exposes public services
Firewall rules	blocks unauthorized traffic
Backend isolation	protects internal services
Network segmentation	separates LAN A and LAN B

Only the public gateway service is accessible from the Internet, while internal microservices remain protected.

15. Conclusion

This laboratory demonstrated the deployment of a secure enterprise-style network using containerized infrastructure.

The implemented architecture successfully achieved the following objectives:

- Secure microservice exposure
- Network address translation
- Firewall-based traffic control
- Internet connectivity
- WAN monitoring capability

The final system accurately simulates a real-world enterprise network architecture where public services are exposed in a controlled manner while internal services remain protected from external access.